

Algorithm:

1. **Import Libraries:** Import OpenCV and necessary modules.
2. **Initialize Video Capture:** Use `cv2.VideoCapture()` to read from video or webcam.
3. **Create MultiTracker Object:** Use `cv2.MultiTracker_create()` to handle multiple objects.
4. **Select ROIs (Regions of Interest):** Allow the user to select objects manually from the first frame.
5. **Add Trackers:** For each ROI, initialize an object tracker (e.g., KCF, CSRT, etc.).
6. **Loop Through Frames:**
 - Read each frame.
 - Update all trackers with the new frame.
 - Draw bounding boxes for each tracked object.
7. **Display Output:** Show the updated frames with tracked object boxes.

Code:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow # For Google Colab

# Download the configuration, weights, and class names files if they don't
exist
!wget
https://raw.githubusercontent.com/pjreddie/darknet/master/cfg/yolov3.cfg
!wget https://pjreddie.com/media/files/yolov3.weights
```

```

!wget
https://raw.githubusercontent.com/pjreddie/darknet/master/data/coco.names
# Download coco.names

# Load YOLO model and COCO class labels
yolo_net = cv2.dnn.readNet("yolov3.weights", "yolov3.cfg") # Updated to
use the downloaded file
layer_names = yolo_net.getLayerNames()
output_layers = [layer_names[i - 1] for i in
yolo_net.getUnconnectedOutLayers()]

# Load class labels
with open("coco.names", "r") as f:
    classes = f.read().strip().split("\n")

# Load an image
image = cv2.imread("/content/istockphoto-159406920-612x612.jpg") #
Replace with your image file
height, width = image.shape[:2]

# Convert image to YOLO format
blob = cv2.dnn.blobFromImage(image, scalefactor=1/255.0, size=(416, 416),
swapRB=True, crop=False)
yolo_net.setInput(blob)
layer_outputs = yolo_net.forward(output_layers)

# Process detections
boxes, confidences, class_ids = [], [], []
for output in layer_outputs:
    for detection in output:
        scores = detection[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]

        if confidence > 0.5: # Confidence threshold
            box = detection[:4] * np.array([width, height, width, height])
            center_x, center_y, w, h = box.astype("int")
            x = int(center_x - (w / 2))
            y = int(center_y - (h / 2))

```

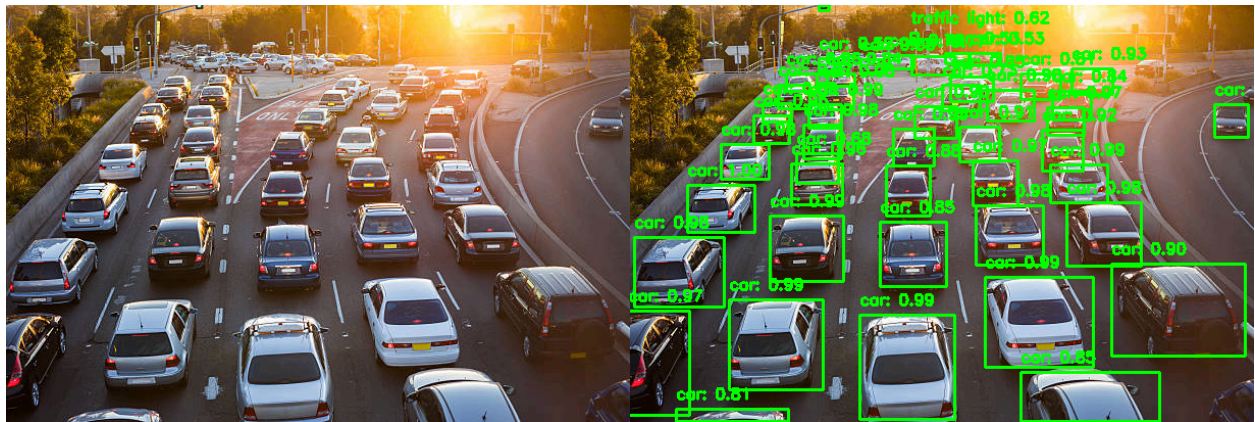
```
        boxes.append([x, y, int(w), int(h)])
        confidences.append(float(confidence))
        class_ids.append(class_id)

# Apply Non-Maximum Suppression (NMS) to remove redundant boxes
indices = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)

# Draw bounding boxes and labels
for i in indices.flatten():
    x, y, w, h = boxes[i]
    label = f"{classes[class_ids[i]]}: {confidences[i]:.2f}"
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
    cv2.putText(image, label, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(0, 255, 0), 2)

# Show the result
cv2.imshow(image)
```

Output:



Result:

Successfully implemented real-time multiple object tracking using OpenCV. Each object was tracked with a unique bounding box across video frames.

